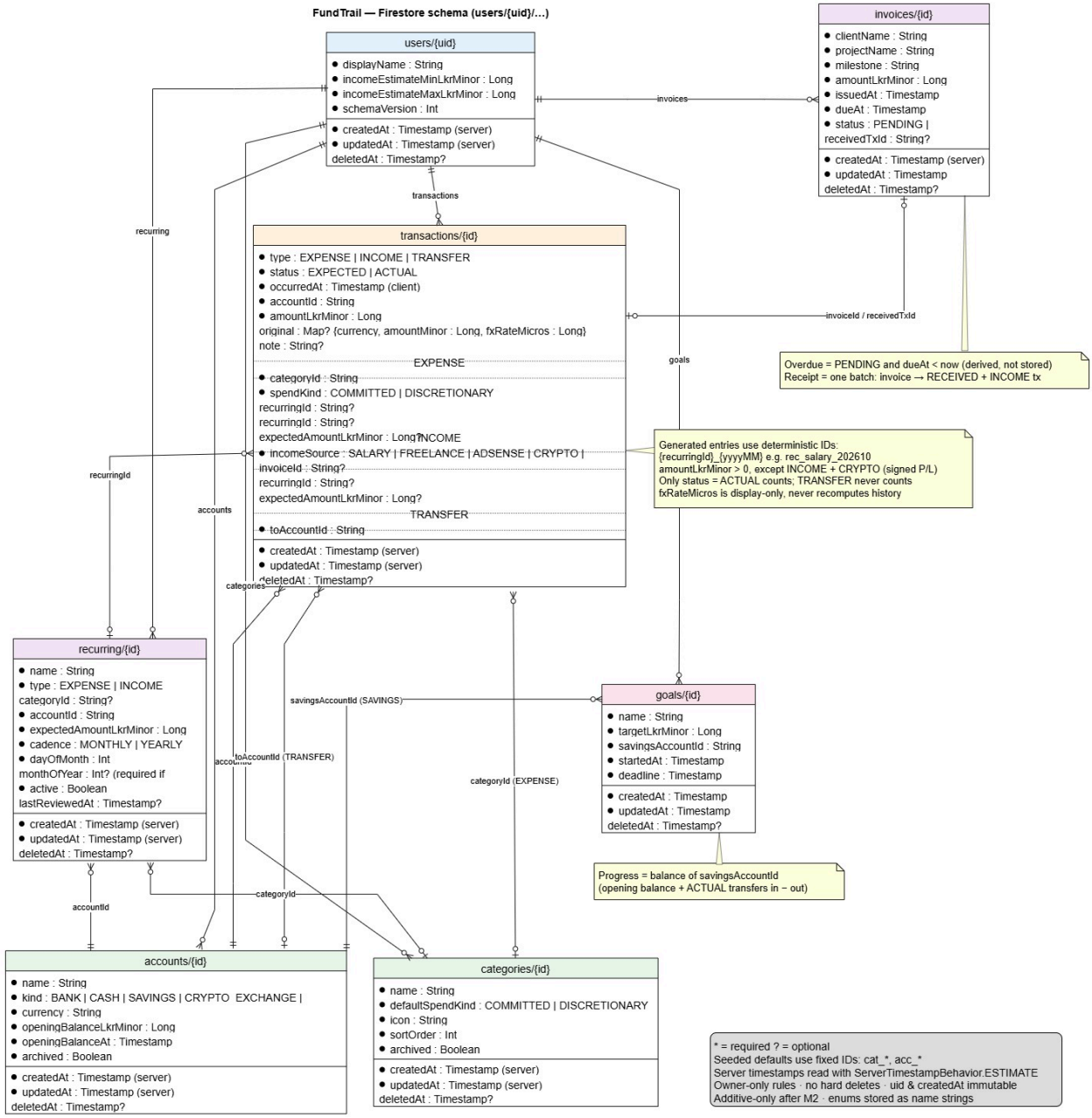
FUNDTRAIL — FIRESTORE SCHEMA

Issue: EPIC-003 / T2 (#17) — sub-tasks S1 collections & fields (#18), S2 multi-currency model (#19), S3 composite index plan (#20) **Use:** Every feature reads and writes data exactly as defined here. Changing this after M2 starts means migrating live data, so §8 (evolution rules) applies from M2 onward. **Related:** *Docs/architecture.md* (#16) for layering; offline and sync behaviour is finalised in #22; security rules are written in #21.

---

1. SCHEMA DIAGRAM



---

2. COLLECTION TREE

All data lives under the signed-in user. There are no top-level shared collections.

```
users/{uid}
 ├── accounts/{id}
 ├── categories/{id}
 ├── transactions/{id}      one collection; `type` discriminates EXPENSE | INCOME |
TRANSFER
 ├── invoices/{id}
 ├── recurring/{id}
 └── goals/{id}
```

Income and expenses share one *transactions* collection with a type discriminator, not separate collections, so a month summary or per-account history is one query over one collection.

---

3. COLLECTIONS AND FIELDS

Legend: Req = required on every document of that kind; **Opt** = may be absent. Types are Firestore types as seen from Kotlin: *String*, *Long* (int64), *Int*, *Boolean*, *Timestamp*, *Map*. Enums are stored as their name string.

3.0 Fields on every document

Field	Type	Req/Opt	Notes
createdAt	Timestamp	Req	*serverTimestamp()* on create. Immutable afterwards.
updatedAt	Timestamp	Req	*serverTimestamp()* on every write.
deletedAt	Timestamp	Opt	Set instead of deleting (soft delete, §6.4).

3.1 *users/{uid}* — profile

Field	Type	Req/Opt	Notes
displayName	String	Req	
incomeEstimateMinMinor	Long	Opt	The user's own monthly income estimate (low end), LKR minor units, shown beside actual income (A-16). Renamed from *incomeEstimateMinLkrMinor* (M3, EPIC-017/T1): optional until set in onboarding, not required on every profile.

Field	Type	Req/Opt	Notes
incomeEstimateMaxMinor	Long	Opt	High end of the estimate, LKR minor units. ≥ min when both are set.
expectedSalaryMinor	Long	Opt	Expected monthly salary, LKR minor units (A-14).
salaryPayday	Int	Req	Day of month the salary is expected, 1–28. Default 25.
schemaVersion	Int	Req	Schema version of this user's data. Only place a version is stored (§8).

The profile document doesn't exist until the user's first settings write (M3, EPIC-012/T1): nothing creates it at sign-up. *SettingsRepository.update()* checks Room for an existing row; if there's none and the initial sync has completed, it creates the document with *displayName* (from the signed-in Firebase user) and *schemaVersion*; otherwise it merges into the existing document. *displayName* and *schemaVersion* are never touched again by *update()* — only the four settings fields are.

3.2 *accounts/{id}*

Field	Type	Req/Opt	Notes
name	String	Req	e.g. "Main bank", "Cash", "MacBook fund"
kind	Enum	Req	*BANK* | *CASH* | *SAVINGS* | *CRYPTO_EXCHANGE* | *OTHER*
currency	String	Req	ISO-style code of the account's own currency (e.g. *LKR*, *USDT*).
openingBalanceLkrMinor	Long	Req	Balance when tracking started. Kavindu's LKR 11,200 goes here on his savings account.
openingBalanceAt	Timestamp	Req	Only transactions on or after this time affect the balance.
archived	Boolean	Req	Hidden from pickers, kept for history.
sortOrder	Int	Req	Order in account pickers.

3.3 *categories/{id}*

Field	Type	Req/Opt	Notes
name	String	Req	User-controlled (A-11).
defaultSpendKind	Enum	Req	*COMMITTED* | *DISCRETIONARY* (A-18). Copied onto each expense at entry.

Field	Type	Req/Opt	Notes
icon	String	Req	Icon key.
sortOrder	Int	Req	Order in the quick-add chips.
archived	Boolean	Req	

3.4 *transactions/{id}*

Common fields (all types):

Field	Type	Req/Opt	Notes
type	Enum	Req	*EXPENSE* | *INCOME* | *TRANSFER*
status	Enum	Req	*EXPECTED* | *ACTUAL*. Only *ACTUAL* counts toward totals (§6.2). Transfers are always *ACTUAL*.
occurredAt	Timestamp	Req	Set by the **client**, never a server timestamp — back-dating is allowed (A-03).
accountId	String	Req	Account the money came from or went to. For a transfer, the **source** account.
amountLkrMinor	Long	Req	The stored truth, in LKR minor units (§4). Positive, except signed crypto results (§6.1).
original	Map	Opt	Present only when the entry was in another currency (§4).
original.currency	String	Req if *original*	e.g. *USD*, *USDT*
original.amountMinor	Long	Req if *original*	Amount in that currency's minor units.
original.fxRateMicros	Long	Req if *original*	LKR per 1 unit × 10^6. Display only.
note	String	Opt	Free text, merchant or memo. Not indexed (§7).

EXPENSE only:

Field	Type	Req/Opt	Notes
categoryId	String	Req	→ *categories/{id}*
spendKind	Enum	Req	*COMMITTED* | *DISCRETIONARY*. Copied from the category at entry; the user may override per entry.

Field	Type	Req/Opt	Notes
recurringId	String	Opt	→ *recurring/{id}* when generated from a template.
expectedAmountLkrMinor	Long	Opt	The expected amount, kept when an *EXPECTED* entry is confirmed with a different actual amount (e.g. rent + variable utilities, A-22).

INCOME only:

Field	Type	Req/Opt	Notes
incomeSource	Enum	Req	*SALARY* | *FREELANCE* | *ADSENSE* | *CRYPTO* | *OTHER*
invoiceId	String	Opt	→ *invoices/{id}* when this income settles an invoice.
recurringId	String	Opt	→ *recurring/{id}* (e.g. salary on the 25th).
expectedAmountLkrMinor	Long	Opt	Expected amount kept after confirming (e.g. salary with a variable bonus, A-12).

TRANSFER only:

Field	Type	Req/Opt	Notes
toAccountId	String	Req	Destination account. Must differ from *accountId*.

3.5 *invoices/{id}*

Field	Type	Req/Opt	Notes
clientName	String	Req	
projectName	String	Req	
milestone	String	Opt	
amountMinor	Long	Req	Invoiced amount, in *currency* minor units. Renamed from *amountLkrMinor* (M3, EPIC-017/T1): invoices can now be raised in USD/USDT, not just LKR.
currency	String	Req	*LKR* | *USD* | *USDT* (§4.2). Added M3.
issuedAt	Timestamp	Req	
dueAt	Timestamp	Req	Drives the follow-up reminder (A-13).

Field	Type	Req/Opt	Notes
status	Enum	Req	*PENDING* | *RECEIVED*. **Overdue is not stored** — it is *PENDING* and *dueAt* < now.
receivedTxId	String	Opt	→ the *INCOME* transaction that settled it. Set when *RECEIVED*. That transaction keeps *amountLkrMinor* as ground truth plus *original* (currency, amount, locked *fxRateMicros*), same as any other foreign-currency entry (§4, ADR-0003).
receivedAt	Timestamp	Opt	Set alongside *receivedTxId*.
lastNotifiedAt	Timestamp	Opt	Last overdue reminder sent (A-13).
note	String	Opt	

3.6 *recurring/{id}*

Field	Type	Req/Opt	Notes
name	String	Req	e.g. "Rent", "Subscriptions", "Gym"
categoryId	String	Req	EXPENSE only in M3 (no *type*/*INCOME* templates); *TODO(M4)* for recurring income.
accountId	String	Req	
expectedAmountMinor	Long	Req	LKR minor units. Renamed from *expectedAmountLkrMinor* (M3, EPIC-017/T1).
isVariableAmount	Boolean	Req	When true, the confirm screen lets the actual amount differ from *expectedAmountMinor* (e.g. variable utility bills).
cadence	String	Req	*WEEKLY*, *MONTHLY* or *YEARLY* (M4, #330); defaults to *MONTHLY* when missing or unrecognised, so a template written before this field existed keeps its old behaviour.
dayOfMonth	Int	Req	1–28 for *MONTHLY*/*YEARLY*; 1–7 (Monday–Sunday, ISO-8601) for *WEEKLY* (M4, #330) — no 29/30/31, so it never needs clamping.
nextDueDate	Timestamp	Req	**Stored**, not derived (M3, EPIC-017/T1, reversing the M1 design below) — *confirm*/*skip* need a cursor to advance from. **Known risk:** a stored cursor can drift across devices if both write offline before syncing; revisit if that surfaces as a bug.

Field	Type	Req/Opt	Notes
isPaused	Boolean	Req	Renamed from *active* (inverted sense) (M3, EPIC-017/T1). A paused template is never due.
lastReviewedAt	Timestamp	Opt	Last "still using this?" review for subscriptions (A-17).

~~The next due date is **derived** from *cadence*, *dayOfMonth* and *monthOfYear*, not stored.~~ Superseded above (M3): *nextDueDate* is stored so *confirm*/*skip* have a cursor to advance by one cadence period.

3.7 *goals/{id}*

Field	Type	Req/Opt	Notes
name	String	Req	e.g. "MacBook Pro M4"
targetLkrMinor	Long	Req	49,000,000 for LKR 490,000.
baselineSavedMinor	Long	Req	Amount already saved when the goal started, LKR minor units. Replaces *savingsAccountId* (M3, EPIC-017/T1): no single account is pinned, so progress tracking moves from "balance of one account" to "saved since baseline" — the exact derivation (which transactions count) is defined by the goal projection use case (#162), not stored here.
startedAt	Timestamp	Req	Client-set once at creation, never updated afterwards; distinct from *createdAt* (§3.0), which can read back null while the create is still pending offline. Goal progress is measured from it (§6.3). Normally the creation time; the debug demo seeder back-dates it so the seeded goal has complete months of pace.
deadline	Timestamp	Req	Stored at UTC midnight of the chosen date (*LocalDate* has no timezone of its own), and read back the same way, so the stored day never shifts with the device's timezone.
isActive	Boolean	Req	Only one active goal per user (M3, EPIC-012/T1): saving a new active goal and archiving the previous one is written in one batch.

~~Goal progress is the balance of *savingsAccountId*.~~ Superseded above (M3): see §6.3.

---

4. MULTI-CURRENCY MODEL

4.1 Fields

Field	Meaning
amountLkrMinor	LKR actually received or spent, in minor units. **The stored truth.** All totals use only this.
original.currency	Currency the user entered (*USD*, *USDT*, …).
original.amountMinor	Amount in that currency, in its minor units.
original.fxRateMicros	Rate used at entry: LKR per 1 unit × 10^6. **Display only.**
status	*EXPECTED* while the LKR figure is an estimate; *ACTUAL* once the credited amount is known.

These replace the field names drafted in #19: *amountMinor* / *currency* / *fxRateToLkr* live inside *original* (rate renamed *fxRateMicros* to make the scaling explicit), and *fxStatus* (*ESTIMATED* | *ACTUAL*) is folded into the generic *status* (*EXPECTED* | *ACTUAL*), which also covers expected salary and rent.

4.2 Currency decimal places

Fixed per currency in **one** code table; never stored per document.

Currency	Decimal places	Minor units per 1
LKR	2	100
USD	2	100
USDT	2	100

4.3 Examples

Entry	***original***	***amountLkrMinor***	***status***
Coffee, LKR 850.00	—	*85000*	*ACTUAL*
AdSense estimate, USD 50.00 at 298.45	*{USD, 5000, 298450000}*	*1492250* (LKR 14,922.50)	*EXPECTED*
Same AdSense, credited LKR 14,870.00	*{USD, 5000, 298450000}*	*1487000*	*ACTUAL*
Crypto realised loss, USDT −12.40	*{USDT, -1240, 300100000}*	*-372124*	*ACTUAL*

The AdSense entry is saved as *EXPECTED* with an estimate, then flipped to *ACTUAL* with the credited LKR amount. The rate stays as entered; the credited LKR amount is what counts.

4.4 Crypto

- Crypto is *INCOME* with *incomeSource* = *CRYPTO*, recording **realised** profit or loss only (A-15). Unrealised holdings are not tracked.
- Results are recorded in USDT, never raw coin quantities (ETH's 18 decimals would overflow a *Long* above ~9.2 ETH).
- A loss is a negative *amountLkrMinor*. Moving money into or out of Binance is a *TRANSFER*, not income (A-21).

4.5 Rationale

- **Why minor units in a *Long*:** floating-point types can't represent most decimal amounts exactly, so sums drift ($0.1 + 0.2 \neq 0.3$). Integer minor units add exactly. No floats for money, anywhere.
- **Why the LKR amount is frozen at entry:** the LKR value is what actually reached the account. Recomputing history with a newer rate would change past months' totals and the goal progress after the fact. Freezing it makes historical totals stable; the rate is kept only to show how the figure was reached.

---

5. DOCUMENT IDS

Documents	**ID**	**Why**
Entries generated from a recurring template	*{recurringId}_{yyyyMM}* of the due month, e.g. *rec_salary_202610*	Two devices (or a reinstall while offline) generating the same month write the **same** document instead of duplicates. Also how the app checks what has already been generated.
Default categories seeded at onboarding	Fixed: *cat_ride_hailing*, *cat_food_delivery*, *cat_coffee_dining*, *cat_groceries*, *cat_subscriptions*, *cat_rent*, …	Seeding twice is idempotent.

Documents	ID	Why
Default accounts seeded at onboarding	Fixed: *default-main-bank*, *default-secondary-bank*, *default-cash*, *default-binance*, *default-other*	Same.
Everything else	Firestore auto-ID	

---

6. INVARIANTS

6.1 Amounts

- *amountLkrMinor* > 0, **except** *INCOME* with *incomeSource* = *CRYPTO*, which may be negative.
- If *original* is present, all three of its fields are present.
- *TRANSFER*: *toAccountId* ≠ *accountId*, and *status* = *ACTUAL*.

6.2 What counts toward totals

- Only *status* = *ACTUAL* counts toward the month summary, actual income and actual saving (Scenario analysis, Lens 1 / A-16).
- *EXPECTED* entries and *PENDING* invoices are shown separately as upcoming or pending.
- *TRANSFER* never counts as income or expense (A-21).
- Documents with *deletedAt* set never count.

6.3 Account balance and goal progress

```
balance(account) = openingBalanceLkrMinor
                 + Σ ACTUAL INCOME   where accountId   = account
                 + Σ ACTUAL TRANSFER where toAccountId = account
                 − Σ ACTUAL EXPENSE  where accountId   = account
                 − Σ ACTUAL TRANSFER where accountId   = account
                 (only occurredAt ≥ openingBalanceAt, deletedAt absent)

goal progress    = goal.baselineSavedMinor + (actual pace accumulated since the goal started, ADR-0006)
```

M3 (EPIC-017/T1) drops *goal.savingsAccountId*: *goals/{id}* no longer pins one account, so progress is no longer a single account's balance. *baselineSavedMinor* is what Kavindu had

already saved when the goal started; the projection use case (#162) adds to it whatever it counts as "actual pace" since then. Kavindu can still earmark money in a dedicated *SAVINGS* account if he wants to; the goal itself just no longer requires it.

6.4 Writes

- **Invoice receipt is one batch:** the invoice update (*status* = *RECEIVED*, *receivedTxId*) and the new *ACTUAL INCOME* transaction (*invoiceId*) are written together or not at all.
- **Recurring confirm is one batch (M3, EPIC-017/T1):** the new *ACTUAL* transaction (*recurringId*) and advancing *nextDueDate* by one month are written together or not at all.
- **Recurring skip (M3, EPIC-017/T1):** advances *nextDueDate* by one month; no transaction is written.
- **Soft delete only:** deleting sets *deletedAt* (for undo and an audit trail). Hard deletes are forbidden by the rules. When the sync listener sees *deletedAt* set, it deletes the Room row; a hard delete arriving as *REMOVED* is handled the same way.
- *createdAt* and the document's owner never change after creation.

6.5 Timestamps

- *createdAt* / *updatedAt* use *serverTimestamp()*. While a write is pending (always, offline), they read back as *null* by default. Snapshots are therefore read with *ServerTimestampBehavior.ESTIMATE*, and mappers must be null-safe, so a pending write never crashes a NOT NULL Room column.
- *occurredAt* is always client-set (back-dating, A-03).

---

7. INDEX PLAN

7.1 Strategy

- **The app reads from Room, not Firestore** (*Docs/architecture.md*). Firestore is queried only by the sync listeners, which listen to each subcollection **unfiltered**. There is no *updatedAt* filter: after the first snapshot only changes arrive, and Firestore resumes incrementally after a reconnect (*Docs/architecture.md* §4.3).
- A composite index is only needed when a query filters on one field and orders by another. The app runs no such query, so **no composite index is used at runtime and none is created**. Unused composites cost write latency and storage.
- The transactions listener is not windowed (no *occurredAt >= cutoff*). Month windows, sums, joins and ordering are Room queries (*TransactionDao*, *AccountDao*, *CategoryDao*). If a windowed listener is ever added, it filters on a single field and uses the automatic single-field index, so it still needs no composite.
- Free-text fields are exempted from single-field indexing, since they are never queried. Short labels (*name*, *kind*, *status*) stay indexed: they are cheap and a future filter or ordering on them would work without a deploy.

- A composite index is added only when a real Firestore query needs one; the Firestore error message gives its exact definition, which is then added here and to the file below.

7.2 Indexes

Index	Query it serves
Automatic single-field on every non-exempt field	Sync listener per subcollection (unfiltered)
(no composites)	—

7.3 *firestore.indexes.json*

The index definitions live in *firestore.indexes.json* at the repo root and are deployed with *firebase deploy --only firestore:indexes* (see the README's Firebase setup section). That file is the source of truth; when this plan changes, change the file in the same PR.

It declares no composite indexes and exempts the free-text fields (*transactions.note*, *invoices.clientName*, *invoices.projectName*, *invoices.milestone*, *invoices.note*) from single-field indexing. An empty *indexes* array in a field override is what exempts a field.

7.4 Query inventory (M3)

Every Firestore read and write the app performs, and whether it needs an index. ... stands for *users/{uid}*.

Query	Path	Filter / order	Index needed	Why
Listen (sync)	*.../transactions*	none	No	Whole user-scoped collection; no *where* or *orderBy*. Not windowed.
Listen (sync)	*.../accounts*	none	No	Same.
Listen (sync)	*.../categories*	none	No	Same.
Listen (sync)	*.../invoices*	none	No	Same.
Listen (sync)	*.../goals*	none	No	Same.
Listen (sync)	*.../recurring*	none	No	Same. Due items are derived from *nextDueDate* in the domain, not queried.

Query	Path	Filter / order	Index needed	Why
Listen (sync)	*users/{uid}* (profile)	single document	No	A document listener, not a query.
Create	*.../{collection}/{id}*	by document ID (*set*)	No	Client-generated ID (§5), no query.
Full update	*.../{collection}/{id}*	by document ID (*set* merge, or *update* for invoices)	No	Sends all mapped domain fields and never *createdAt* (§9).
Partial update	*.../{collection}/{id}*	by document ID	No	Sends only the supplied fields plus *updatedAt*.
Soft delete	*.../{collection}/{id}*	by document ID (*deletedAt*, *updatedAt*)	No	Hard deletes are denied by the rules (§6.4).
Save goal	*.../goals/{id}* and the archived goal	batch of two document *set*s	No	Keeps one active goal (§3.7); writes by ID only.
Confirm recurring	*.../transactions/{recurringId}_{yyyyMM}* and *.../recurring/{id}*	batch: transaction *set* + template *update* of *nextDueDate*	No	Atomic confirm (§6.4); writes by ID only. Skip is a single *update*.
Mark invoice received	*.../invoices/{id}* and *.../transactions/{id}*	batch: invoice *update* + transaction *set*	No	Atomic settle (§6.4); writes by ID only.
Profile update	*users/{uid}*	by document ID (*set* merge)	No	Single document.

If a collection gains a filtered query, add a row here and to *firestore.indexes.json* in the same PR.

---

8. EVOLUTION RULES (FROM M2 ONWARD)

- **Additive only.** Never rename or retype a field; add a new one instead.

- **Every new field is optional**, with a default in the mapper.
- **Enums are stored as name strings**, never ordinals. An unknown value maps to a fallback, so an older app version doesn't crash on a newer value.
- *schemaVersion* lives on the user document only; per-document versions aren't needed at this scale.

---

9. SECURITY CONSTRAINTS (IMPLEMENTED IN #21, FIELD VALIDATION IN #153/#154)

- Owner-only: every read and write under *users/{uid}* requires *request.auth.uid == uid*.
- The profile document (*users/{uid}*) and each collection below it (*accounts*, *categories*, *transactions*, *invoices*, *recurring*, *goals*) is matched explicitly, each with its own validation function — not the single recursive *users/{uid}/{path=**}* wildcard used before M3. Rules v2 ORs every matching rule, so a wildcard match can't be narrowed by adding stricter rules underneath it; it has to be split into one match per collection instead.
- Every collection's *create* and *update* run the same validator: *hasOnly([...])* rejects any field not in §3, every required field is checked for presence and type, and every enum field is checked against its allowed values (§3's "Req/Opt" column and the per-type notes). An optional field may be absent **or** explicitly *null* — the app's *toMap()* always sends unset optionals as *null* rather than omitting the key, so the rule has to accept both.
- Collection-specific invariants (§6.1, §3.5): *TRANSFER* requires *toAccountId != accountId* and *status == 'ACTUAL'*; *EXPENSE* requires *categoryId* and *spendKind*; *INCOME* requires *incomeSource*; *amountLkrMinor > 0* unless *INCOME* with *incomeSource == 'CRYPTO'*; an invoice's *status* can move *PENDING → RECEIVED* only once (the second attempt — e.g. a second device settling the same invoice — is denied), closing the gap the batch write in *InvoiceRemoteDataSource.markReceived* can't close itself (a batch can't read before it writes).
- *allow delete: if false* on every collection — soft delete only.
- *createdAt* is required on create and immutable on update, so an update must send the stored value unchanged (or merge without it), never a new *serverTimestamp()*. *displayName* and *schemaVersion* on the profile document, and *startedAt* on a goal, are likewise write-once (§3.1, §3.7).
- Everything outside *users/{uid}* is denied; everything under it that isn't one of the six collections above is denied too (no more open-ended wildcard).
- **Out of scope (#153):** App Check. Rules validate *shape*, not *origin* — anyone with a valid Firebase Auth session can still call the API from a non-app client. App Check would attest the request came from the real app; it's flagged here as explicit future work, not implemented.
- Rules live in *firestore.rules* at the repo root; field-level tests in *rules-tests/* (emulator, *Docs/testing/rules.md*).
- Deployed with *firebase deploy --only firestore:rules* on 2026-10-22 (M3, #153/#154). The M2 owner-only Rules Playground checks for accounts, categories and transactions (#122) still apply; field validation is additionally covered by *rules-tests/test/rules.test.js*.